Assignment

1. What is React and why is it used?

React is a JavaScript library for building user interfaces, especially single-page applications. It's used because it enables reusable components, efficient UI updates via the Virtual DOM, and a declarative style that makes UI code easier to reason about and maintain.

2. What is JSX? One advantage?

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like code inside JavaScript. Advantage: it makes UI structure easier to read and write, since markup and logic live together, and it gets compiled to `React.createElement()` calls under the hood.

3. Props vs State

- **Props**: Data passed *into* a component from its parent. Read-only; the component receiving them cannot modify them.
- **State**: Data managed *within* a component itself. It's mutable (via setter functions) and changes trigger re-renders.

In short: props flow down from outside, state is internal and controlled by the component.

4. Purpose of `useState()`? Example

`useState()` lets functional components hold and update local state.

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
```

```
    </button>
  );
}
```

5. What is `useEffect()`? When used?

`useEffect()` handles side effects in functional components — things outside pure rendering, like API calls, subscriptions, timers, or manually updating the DOM. It runs after render, and you can control when it re-runs using a dependency array.

```
useEffect(() => {
  fetchData();
}, [dependency]);
```

Commonly used for: data fetching, event listeners, cleanup on unmount.

6. What is the Virtual DOM? How does it improve performance?

The Virtual DOM is a lightweight, in-memory representation of the actual DOM. When state changes, React creates a new Virtual DOM tree, compares it (diffing) with the previous one, and updates only the changed parts in the real DOM (reconciliation). This avoids costly full-page re-renders and makes updates faster.

7. Why is the `key` prop important in lists?

`key` gives each list item a stable identity, helping React efficiently track which items changed, were added, or removed during re-renders. Without proper keys (or using array indexes carelessly), React may re-render items incorrectly or inefficiently, causing bugs especially with dynamic lists.

8. Purpose of `useRef()`? Example use case

`useRef()` creates a mutable reference that persists across renders without causing re-renders when updated. Common use case: accessing/manipulating DOM elements directly.

```
function TextInput() {
  const inputRef = useRef(null);

  const focusInput = () => {
```

```
    inputRef.current.focus();
  };

  return (
    <>
      <input ref={inputRef} type="text" />
      <button onClick={focusInput}>Focus Input</button>
    </>
  );
}
```

9. `useMemo()` vs `useCallback()`

- **`useMemo()`**: Memoizes a *computed value*, recalculating it only when dependencies change. Useful for expensive calculations.
- **`useCallback()`**: Memoizes a *function itself*, preventing it from being recreated on every render. Useful when passing callbacks to child components to avoid unnecessary re-renders.

```
const memoizedValue = useMemo(() => computeExpensive(a, b), [a, b]);
const memoizedFn = useCallback(() => doSomething(a), [a]);
```

10. What is the Context API? When to use it?

The Context API allows sharing data (like themes, user auth, language settings) across multiple components without manually passing props down through every level ("prop drilling"). Use it when several components at different nesting levels need access to the same data, and passing props through each intermediate component becomes cumbersome.